

ADVANCED WEB TECHNOLOGIES

Purpose statement

This module describes the skills, knowledge and attitude required to develop modern web applications using advanced web technologies. This module is intended for students pursuing TVET advanced diploma in Information Technology. At the end of this module, the students will be able to apply OOP principals for PHP, connect to MySQL database using PHP, apply frameworks and content management systems, and perform data formats and exchange using XML and AJAX, as well as deploying the web application on the web server.

LEARNING UNIT 1 – APPLY OOP WITH PHP

Learning Outcome 1.1: Explain OOP Concepts

Key Terms

1. Class:

It is a blueprint for creating objects. It defines properties (attributes) and methods (behaviors).

Example:

```
class Car {  
    // Properties  
    public $brand = "Toyota";  
    public $color = "Red";  
  
    // Method  
    public function start() {  
        echo "Car is starting\n";  
    }  
}
```

2. Object:

An instance of a class.

Example:

```
$myCar = new Car(); // Creating an object of the Car class  
echo $myCar->brand; // Accessing a property  
$myCar->start(); // Calling a method
```

3. Access Modifier:

It specifies the accessibility of class members: public, protected, and private.

Example:

```
class BankAccount {
    private $balance;

    public function __construct($balance) {
        $this->balance = $balance; // Private property
    }

    public function getBalance() { // Public method
        return $this->balance;
    }

    private function hiddenMethod() { // Private method
        echo "This is hidden";
    }
}

$account = new BankAccount(1000);
echo $account->getBalance(); // Accessible
// echo $account->balance;    // Error: Cannot access private property
```

4. Properties:

They are attributes that belong to a class.

Example:

```
class Person {
    public $name;
    public $age;
}

$person = new Person();
$person->name = "John";
$person->age = 30;
echo "Name: $person->name, Age: $person->age";
```

5. Methods:

They are functions defined within a class that operate on the properties behaviors of the class.

Example:

```
class Calculator {
    public function add($a, $b) {
        return $a + $b;
    }
}

$calc = new Calculator();
```

```
echo $calc->add(5, 3); // Output: 8
```

6. Constructor:

A special method called automatically when an object is created.

Example:

```
class User {
    public $name;

    public function __construct($name) {
        $this->name = $name;
        echo "User $name has been created\n";
    }
}

$user = new User("Alice");
```

7. Destructor:

A special method called when an object is destroyed or goes out of scope.

Example:

```
class FileHandler {
    public function __destruct() {
        echo "Object destroyed, cleaning up resources\n";
    }
}

$handler = new FileHandler();
unset($handler); // Destructor is invoked
```

8. Overloading:

Using the same method name but with different parameters (not directly supported in PHP but achievable using magic methods).

Example:

```
class Overloader {
    public function __call($name, $arguments) {
        if ($name == 'add') {
            if (count($arguments) == 2) {
                return $arguments[0] + $arguments[1];
            } elseif (count($arguments) == 3) {
                return $arguments[0] + $arguments[1] + $arguments[2];
            }
        }
    }
}

$obj = new Overloader();
echo $obj->add(1, 2); // Output: 3
echo $obj->add(1, 2, 3); // Output: 6
```

9. Overriding:

A subclass provides its own implementation of a method defined in its parent class.

Example:

```
class ParentClass {
    public function greet() {
        echo "Hello from Parent\n";
    }
}

class ChildClass extends ParentClass {
    public function greet() {
        echo "Hello from Child\n";
    }
}

$child = new ChildClass();
$child->greet(); // Output: Hello from Child
```

10. Namespace:

Organizes code into logical groups to prevent name collisions.

Example:

```
namespace Library;

class Book {
    public function read() {
        echo "Reading a book\n";
    }
}

$book = new \Library\Book(); // Accessing class in a namespace
$book->read();
```

11. Interface:

Defines a contract that implementing classes must follow.

Example:

```
interface Animal {
    public function makeSound();
}

class Dog implements Animal {
    public function makeSound() {
        echo "Woof!\n";
    }
}

$dog = new Dog();
```

```
$dog->makeSound(); // Output: Woof!
```

OOP Features

1. Abstraction:

Hiding implementation details and exposing only essential features.

Example:

```
abstract class Vehicle {  
    abstract public function move(); // Abstract method  
}  
  
class Car extends Vehicle {  
    public function move() {  
        echo "Car is moving\n";  
    }  
}  
  
$car = new Car();  
$car->move(); // Output: Car is moving
```

2. Encapsulation:

Restricting access to certain properties or methods.

Example:

```
class Employee {  
    private $salary;  
  
    public function setSalary($amount) {  
        if ($amount > 0) {  
            $this->salary = $amount;  
        }  
    }  
  
    public function getSalary() {  
        return $this->salary;  
    }  
}  
  
$emp = new Employee();  
$emp->setSalary(5000);  
echo $emp->getSalary(); // Output: 5000
```

3. Inheritance:

A class can inherit properties and methods from another class.

Example:

```
class Animal {
    public function eat() {
        echo "Eating...\n";
    }
}

class Cat extends Animal {
    public function meow() {
        echo "Meow...\n";
    }
}

$cat = new Cat();
$cat->eat(); // Output: Eating...
$cat->meow(); // Output: Meow...
```

4. Polymorphism:

Objects can take on multiple forms, usually through method overriding.

Example:

```
class Shape {
    public function draw() {
        echo "Drawing a shape\n";
    }
}

class Circle extends Shape {
    public function draw() {
        echo "Drawing a circle\n";
    }
}

class Square extends Shape {
    public function draw() {
        echo "Drawing a square\n";
    }
}

$circle = new Circle();
$square = new Square();

$circle->draw();
$shape->draw();
$shapes = [new Circle(), new Square()];
foreach ($shapes as $shape) {
    $shape->draw(); // Calls the appropriate draw() method
}
```

Advantages of OOP in PHP

1. **Fast and efficient:**

Reusability of classes and modular structure make OOP programs faster to develop and more efficient.

2. **Easy to use & modify:**

Encapsulation ensures that changes to one part of the program do not affect others.

3. **Easy to maintain & debug:**

Each class is self-contained, making it easier to maintain and debug.

4. **Clear structure for programs:**

Classes and objects provide a clear and logical structure.

5. **Allows re-use of code:**

Inheritance and polymorphism allow reusing existing code, reducing redundancy.

By incorporating these features and concepts, OOP in PHP allows developers to write clean, reusable, and maintainable code.